Vibe Hash: Proof of Useful Work by Verifiable Neural Network Training

Cole Heideman
vibehash.org

Abstract. Proof-of-work secures a ledger by making blocks expensive to produce, but the work itself is thrown away. We propose a network in which the work is the training of neural networks that people pay to have trained. Training is useful, but unlike hashing it cannot be checked at a glance. We divide each job into small, deterministic units of stochastic gradient descent. A worker commits to a Merkle tree over the checkpoints its unit passes through before a public random beacon, unknown until every commitment is in, chooses which segments will be checked. A randomly chosen staked verifier replays those segments and compares the result within a tolerance that absorbs rounding differences between processors but not skipped work. Faking part of a unit is caught with probability at least equal to the part faked, and costs the round's pay and half the cheater's stake, so cheating does not pay whenever honest work does. Every unit that verifies is paid in proportion to its floating-point operations: there is no lottery, and a laptop earns the same rate per operation as the largest machine on the network. Validators replay each round from its signed record before voting on its block, so no single node can bend the rules in its own favour.

1. Introduction

Bitcoin [1] showed that a ledger can be kept without a trusted party by having nodes compete to find hashes below a target. The hashes have no use beyond the competition, and the competition is a lottery: each block's reward goes to one winner, which has pushed mining into pools and onto specialised hardware. Meanwhile, training machine-learning models has become one of the largest uses of computation in the world, while most capable processors, the graphics cards in gaming PCs, laptops and home servers, sit idle most of the day.

What is needed is a way to pay for training the way Bitcoin pays for hashing, without trusting the people who do it. The difficulty is verification. A hash is checked by computing one more hash; a step of training is checked only by doing it again. Work on proof of learning [3] has shown that a training run can be audited by replaying parts of it from saved checkpoints. In this paper we make that audit the core of a consensus protocol. Work is cut into units small enough for a slow laptop to finish within a round. Every unit is committed to before anyone knows which part of it will be checked. The check is a replay assigned at random, and the incentives make an honest unit worth more than a faked one. Because work is checked rather than raced, everyone whose work passes is paid in proportion to it.

2. Jobs and Work Units

A requester posts a job: a model specification, the hash of a dataset, training settings, a fee per round and a budget held in escrow. The network trains the job in rounds. Each round starts from the job's current weights W_r and divides the work into units of the form: starting from W_r, run S steps of stochastic gradient descent with batches chosen by the seed σ, saving a checkpoint every c steps. The batch of each step is derived from σ and the step number alone, so any part of a unit can be replayed on its own.

Units are deliberately small: the slowest eligible machine must finish one before the deadline. A faster machine receives more units, not larger ones, and a unit is never given to a machine without the memory to hold it. A worker's share grows with what it has delivered. A newcomer starts with a few units, may take at most twice as many in the next round if it delivered all of them, and drops back to what it delivered if it misses a deadline, so claiming more speed than one has gains nothing. At the end of the round, the accepted units are averaged into W_{r+1}, as in federated averaging [6]. The requester pays only for the share of each round that verified, and the trained weights are public: every worker receives them each round anyway.

3. Commitments

A unit with a checkpoint every c steps passes through checkpoints C_0, …, C_k, which divide it into k segments. The worker hashes each checkpoint, builds a Merkle tree [2] over the hashes, and before the compute deadline publishes a signed commitment

$$H(\text{ unit} \parallel \text{public key} \parallel \text{Merkle root} \parallel \text{salt }).$$

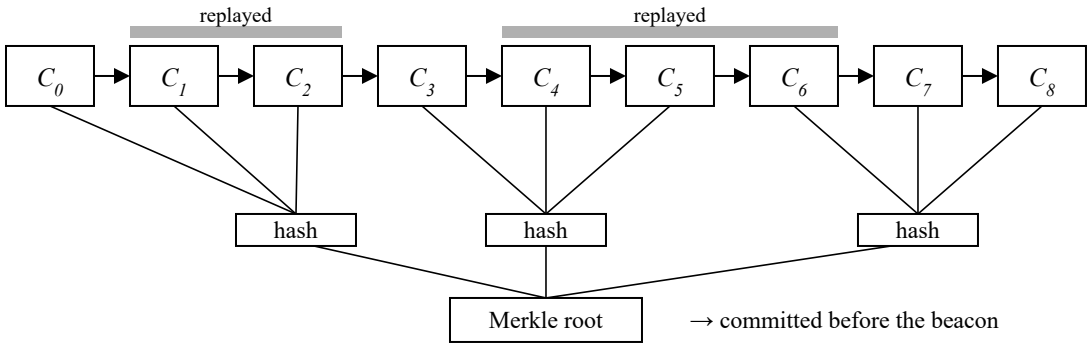


Figure 1: A unit's checkpoints, bound to one Merkle root before the beacon. Here the beacon then picks the shaded segments, $C_1 \to C_2$, $C_4 \to C_5$ and $C_5 \to C_6$, to replay.

The commitment reveals nothing about the result, so it cannot be copied, and it binds the worker to every checkpoint before anyone knows which will be examined. The first checkpoint must hash to the job's current weights, so a unit cannot start from invented ones. Workers upload their checkpoints to the round's node as content-addressed blobs before committing, which makes the reveal that follows a short message and counts the upload as part of the work.

4. The Beacon

After the compute deadline the round's randomness is

$$beacon = H(\text{ previous block} \parallel \text{round secret} \parallel \text{all commitments }).$$

The round secret is chosen by the validator leading the round when it opens it. The secret's hash is published at once, and the secret itself only in the sealed round's record. Without it, a worker who could see every commitment, for instance one alone in a round, could re-salt its own commitment until the beacon chose only segments it had computed honestly. Workers compare the published hash with the record, so the leader cannot change the secret after seeing the commitments either. From the beacon follow, for each unit, which q of its k segments are checked (three of eight by default) and who checks them, and from the secret, where test submissions are planted.

5. Verification

Workers reveal the Merkle root, the salt, the final weights, and the checkpoints at both ends of every challenged segment with their Merkle proofs. A reveal that does not match its commitment is fraud. The checks, cheapest first:

- **Sanity.** The final weights must be finite, must equal the last checkpoint, and must differ from the start by a plausible amount. This stops garbage, and inflated updates meant to poison the model, without any replay.
- **Replay.** The assigned verifier re-runs each challenged segment from C_i and compares the result with C_{i+1}. It must lie within the job's tolerance and within 1% of how far the segment actually moved the weights. The first condition absorbs rounding differences between processors. The second stops a worker from resubmitting untrained weights for a segment whose honest update is smaller than the tolerance, which happens late in training.
- **Replication.** A random fraction of units is secretly given to two workers. Their final weights must agree; if they don't, a full replay decides who lied.
- **Canaries.** The protocol slips known-bad tasks into verifiers' queues: copies of real tasks with one segment's update rescaled. They keep the structure of honest training and their positions come from the round secret, so they cannot be told apart from real work. A verifier that approves one is slashed, so approving without replaying does not pay.
- **Arbitration.** A rejection is a claim, not a verdict. The node replays the disputed segment itself, and a verifier who rejected honest work is slashed for the false accusation.

Committing and then not revealing forfeits the worker's whole round, but it is not slashed, because it cannot be told apart from a machine that crashed or lost its connection. Section 11 shows it is still unprofitable.

6. Rounds and Consensus

Each round produces one block. The block's summary carries the hash of the round's record: the workers' signed registrations, commitments, reveals and verdicts, the round's rules, and the outcome of each check. Every choice made in the round is drawn from public seeds by a deterministic generator: which units each worker gets, which are duplicated, who checks whom, where the canaries go, which job runs. Anyone can therefore run the round again from its record and confirm that the block's summary follows from the signed messages.

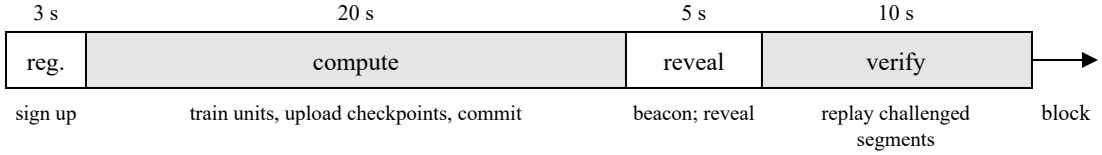


Figure 2: A round with the default phase lengths, drawn to scale. Jobs with larger models get longer phases, sized so a modest computer on a home connection can still move its checkpoints in time.

Validators hold stake and take turns leading rounds, in a stake-weighted order fixed by the block height alone. The leader runs the round with the workers and proposes the block together with its record. Every other validator applies the block and replays the record before voting, and voting follows Tendermint [4]. A block is final once validators holding more than two thirds of the stake have signed it, and those signatures are its commit certificate. A leader that pays itself extra, favours a worker or forges a verdict fails the replay, is voted down, and the next validator in the order leads instead. Rewards and slashes are never taken from the proposer: every node derives them from the round's summary and the chain's state. Useful work decides who is paid; stake decides who votes.

A node signs a receipt for every registration, commitment, reveal and verdict it accepts, naming the message's hash. A receipt for a message missing from its round's record is signed proof that the node dropped it.

7. Incentive

Each round has a pool, made of the block subsidy and the fee the job releases for the work that verified:

$$pool = subsidy(height) + \min(\, escrow, fee \times accepted\ units \,/\, units\ per\ round \,).$$

Eighty percent goes to workers in proportion to their verified training operations, fifteen percent to verifiers in proportion to the operations they replayed, both weighted by reputation, and five percent to the block's proposer. Anything unclaimed is burned rather than given to the proposer, so a proposer gains nothing by rejecting good work. Reputation is a multiplier between 0.5 and 1.0. It starts at 0.8, rises by 0.02 in each round with verified work, falls by 0.05 for a missed deadline, and drops to 0.5 on fraud. It moves per round rather than per unit, so a laptop builds it exactly as fast as a large machine.

Fraud costs half of the offender's stake, including coins still waiting in the 20-round unbonding queue, so a cheater cannot withdraw ahead of the verdict. Half of the slashed stake goes to whoever caught it and half is burned. The subsidy follows Bitcoin's shape stretched to Vibe Hash's block time: 3.125 VIBE a block, halving every 3,320,000 blocks, which is about four years at 38-second rounds, with at most 21,000,000 VIBE ever. Amounts are integers of 10^{-8} VIBE and rounding remainders are burned, so the supply is exactly the genesis allocation plus subsidies minus burns.

Pay per verified operation is the same for every machine. A laptop earns less in total because it does less work, but at the same rate, and every round in which it completes work rather than almost never, so no pool is needed.

8. A Fair Start

Bitcoin needed no stake: a hash proves itself, so anyone could mine from the first block. Work that is only spot-checked needs a bond the cheater stands to lose, and if every stake has to be handed out in the genesis block, the first coins are a premine. Instead, anyone may register without stake, and pay to a worker below the minimum stake of 5 VIBE goes into its stake until it reaches the minimum. Until then the worker takes only units staked workers did not need, and never checks anyone else's work, so a swarm of free identities cannot approve each other's fakes. The first validators vote with bootstrap stake, which can never be withdrawn or spent and is burned if they leave, so it is outweighed as others earn stake and join. Every spendable coin starts from work.

9. Arithmetic Across Devices

Replays must agree across very different hardware. Every checkpoint is rounded to the job's precision before it is hashed, so a 32-bit job's Merkle leaves are the same whether its weights came from a graphics card or a CPU. Workers on graphics cards disable reduced-precision matrix modes and request deterministic kernels. Measured on a consumer NVIDIA graphics card against the reference CPU implementation, replays of segments of 5 to 500 steps on models of 69 thousand to 2.2 million weights differed by 4×10^{-9} to 8×10^{-8}, between 1,250 and 22,000 times inside the tolerance of 10^{-4}, and the same card gave identical bits on every run. The same measurements show why the update condition of Section 5 is needed: one honest 5-step segment of the 2.2-million-weight model moved the weights by only 4.3×10^{-5}, less than the tolerance.

10. Models Larger Than One Computer

A unit as described is the whole model, which limits a job to what one machine can hold. Larger models are split into shards, contiguous blocks of layers, and a unit is trained by a team with one worker per shard, as a pipeline [7]: activations flow forward from shard to shard and gradients flow back. Done in the same order, this is the same arithmetic one machine would do,

and ends at exactly the same bits. Each worker commits to its shard's checkpoints and to every tensor that crossed its edges. A unit's work becomes a grid of cells, each one shard over one segment, and a checker replays a single cell holding only that shard. When two neighbours disagree about what crossed between them, their signed edge messages show which one lied.

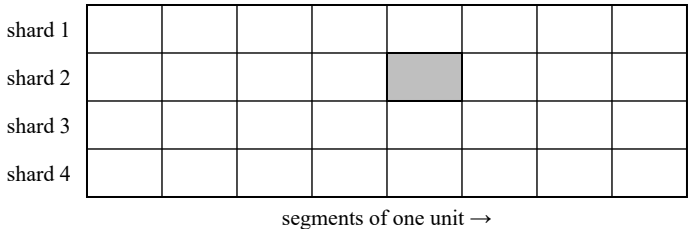


Figure 3: A sharded unit as a grid of cells. A checker replays one cell, here shard 2 over segment 5, holding only that shard.

For the largest open models, the useful job is fine-tuning: the original weights stay frozen and small low-rank adapters are trained [5], which also keeps the checkpoints that move between workers small. As an example, a 763-billion-parameter mixture-of-experts model stored in 510 GB needs, by this method, a team of 40 graphics cards with 12 GB of memory each, 20 with 24 GB or 14 with 32 GB, while its adapters come to about 160 MB. Such teams are limited by their network links rather than their processors, so training is spread across many teams at once. The team protocol works in simulation and is not yet run by the network.

11. Calculations

Suppose a worker skips the training of m of a unit's k segments and makes up their checkpoints. The beacon checks q distinct segments, unknown to the worker when it commits, so the worker escapes only if all q fall among the $k - m$ segments it trained:

$$P(\text{escape}) = \mathrm{C}(k - m, q) \,/\, \mathrm{C}(k, q) \;\le\; (1 - m/k)^{q}.$$

With the defaults $k = 8$ and $q = 3$:

```
from math import comb

def caught(m, k=8, q=3):
    return 1 - comb(k - m, q) / comb(k, q)

for m in range(1, 9):
    print(m, m / 8, round(caught(m), 4))
```

m	*fraction skipped*	*P(caught)*
1	0.125	0.3750
2	0.250	0.6429
3	0.375	0.8214
4	0.500	0.9286
5	0.625	0.9821
6–8	≥ 0.750	1.0000

In every row the probability of being caught is at least the fraction skipped. Replicated units and canaries only add to it. Now let u be the cost of computing one unit, R the worker's pay for the whole round and s its stake. Skipping m/k of a unit saves $(m/k)\cdot u$, and being caught loses $R + s/2$, so cheating pays only if

$$(m/k) \cdot u \;>\; P(\text{caught}) \cdot (\, R + s/2 \,).$$

Since $P(\text{caught}) \geq m/k$, that requires $u > R + s/2$: one unit would have to cost more than the pay for the whole round plus half the stake. If that were so, no one would mine honestly either. A worker that commits, sees which segments are challenged and withholds its reveal avoids the slash but loses R with the same probability, so backing out pays only if $u > R$, which is likewise a network nobody would mine on.

12. Conclusion

We have proposed a system that pays for the training of neural networks without trusting those who do it. Work is divided into deterministic units, each committed to through a Merkle tree before a beacon nobody can predict chooses which segments are replayed, and a randomly chosen verifier repeats them within a tolerance that admits honest rounding but not skipped work. Stake, reputation and test submissions make cheating, lazy checking and false accusations each cost more than they could gain. Because the work is checked rather than raced, everyone whose work passes is paid in proportion to it, and an old laptop earns the same rate as the largest machine on the network. Validators replay every round from its signed record before agreeing on its block, so the rules hold without a central coordinator. Useful work decides who is paid, and stake decides who votes. A test network implementing this design runs at vibehash.org; it trains small models today, its coin VIBE has no cash value, and the software has not been audited.

References

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.

[2] R. C. Merkle, "Protocols for public key cryptosystems," in Proc. 1980 Symposium on Security and Privacy, IEEE Computer Society, pages 122–133, April 1980.

[3] H. Jia, M. Yaghini, C. A. Choquette-Choo, N. Dullerud, A. Thudi, V. Chandrasekaran and N. Papernot, "Proof-of-Learning: Definitions and Practice," in IEEE Symposium on Security and Privacy, 2021.

[4] E. Buchman, J. Kwon and Z. Milosevic, "The latest gossip on BFT consensus," arXiv:1807.04938, 2018.

[5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," in International Conference on Learning Representations, 2022.

[6] H. B. McMahan, E. Moore, D. Ramage, S. Hampson and B. Agüera y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in Artificial Intelligence and Statistics, 2017.

[7] Y. Huang et al., "GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism," in Advances in Neural Information Processing Systems, 2019.

Disclaimer: All product names, logos, and brands are property of their respective owners and are used purely for identification. Vibe Hash isn't affiliated with or endorsed by them.